Chapter 5. Composing objects

This chapter covers

- Structuring code with classes and inheritance
- Using prototypes for dynamic flexibility
- Extending your program with mixins

In the previous chapter you traveled the road from object literals to prototypes and then classes. Along the way you started to build an understanding of how objects work in CoffeeScript. Now it's time to reverse course by first defining some classes of your own and then slowly taking them apart to see the objects and prototypes that underpin them. This will give you the complete picture of objects in CoffeeScript.

You'll begin the exploration with classical objects and class inheritance, followed by methods and other class properties and an explanation of the `super` keyword. Then you'll learn how to modify objects, classes, and prototypes. Finally, this chapter ends by teaching you what mixins are and how to use them. The flexible syntax and object model of CoffeeScript make writing mixins elegant, even though they're not a built-in language feature. First up, classes and classical objects.

5.1. Being classical

Although objects in CoffeeScript are fundamentally prototype-based, the prototype-based approach that you learned about in chapter 4 is uncommon in programming languages. The class-based approach, where objects are created *using classes*, is common. So if classes are common, they must be good for something, right? Yes, they are. Classes are natural and convenient for declaring common properties and methods to be shared by multiple objects.